Prompt

Write me a program in C++ to implement the following requirements

- The program will prioritize emails for a busy company CEO.
- You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
- You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
- You must create functions from scratch. Do not include pre-existing heap modules.

The program will take a text file input with the following commands to do their specified tasks:

- EMAIL <sender category>, <subject line>, <date>
    - The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows:
        - Boss first
        - Subordinate second
        - Peer third
        - ImportantPerson fourth
        - OtherPerson last
    - If the same sender has sent more than one email, read the NEWEST first
    - EMAIL will be followed by a space, other fields are delimited
    - Assume <sender category> is one of the five strings listed above.
    - Assume <subject line> is a string which may contain spaces, but not commas
    - Assume <date> is in the format: MM-DD-YYYY
- NEXT
    - Next email for the CEO to read. Display the information on the terminal in the following format:
        - Sender:
        - Subject:
        - Date:
- READ
    - The CEO has read the email and dealt with it
- COUNT
    - Display the remaining emails.

Also be sure to handle any situation where the CEO attempts to read or view the next email when there are none. Do not let the program crash. The following is an example of a possible input file with its output

Input:

EMAIL Peer,Can you help me on this?,12-01-2024

EMAIL OtherPerson,Try our product,12-19-2024

EMAIL Boss,Important,12-20-2024

EMAIL Subordinate,How do I handle this?,12-25-2024

EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024

EMAIL Boss,Never Mind,01-03-2025

COUNT

NEXT

READ

NEXT

READ

COUNT

Output:

There are 6 emails to read.

Next email:

Sender: Boss

Subject: Never Mind

Date: 01-03-2025

Next email:

Sender: Boss

Subject: Important

Date: 12-20-2024

There are 4 emails to read.

Claude, accessed through browser, code:

```
// CEO Email Prioritizer
// Uses a hand-written, array-based MaxHeap as a priority queue.
//
// Priority: sender category first (Boss > Subordinate > Peer > ImportantPerson >
// OtherPerson), then newest date first. Ties are broken by arrival order.
//
// Build:  g++ -std=c++11 -o email_priority email_priority.cpp
// Run:    ./email_priority input.txt     (or run with no args and enter a filename)

#include <iostream>
#include <fstream>
#include <string>
#include <cstdio>

using namespace std;

// ---------------------------------------------------------------- Email ----
struct Email {
    string sender;
    string subject;
    string date;      // MM-DD-YYYY (as given)
    int    rank;      // 0 = Boss ... 4 = OtherPerson (lower = more important)
    int    dateKey;   // YYYYMMDD, so bigger = newer
    long   seq;       // arrival order, used to break exact ties
```

```
    Email() : rank(0), dateKey(0), seq(0) {}
};

// Returns true if a should be read before b.
bool higherPriority(const Email& a, const Email& b) {
    if (a.rank != b.rank)      return a.rank < b.rank;       // better category wins
    if (a.dateKey != b.dateKey) return a.dateKey > b.dateKey;  // newest wins
    return a.seq < b.seq;                                   // earlier arrival wins
}

// ------------------------------------------------------------ MaxHeap ----
class MaxHeap {
private:
    Email* data;
    int    count;
    int    capacity;

    static int parent(int i) { return (i - 1) / 2; }
    static int left(int i)   { return 2 * i + 1; }
    static int right(int i)  { return 2 * i + 2; }

    void grow() {
        int newCap = capacity * 2;
        Email* bigger = new Email[newCap];
        for (int i = 0; i < count; i++) bigger[i] = data[i];
        delete[] data;
```

```
    data = bigger;
    capacity = newCap;
}

void swapAt(int i, int j) {
    Email tmp = data[i];
    data[i] = data[j];
    data[j] = tmp;
}

void siftUp(int i) {
    while (i > 0 && higherPriority(data[i], data[parent(i)])) {
        swapAt(i, parent(i));
        i = parent(i);
    }
}

void siftDown(int i) {
    while (true) {
        int best = i;
        int l = left(i), r = right(i);
        if (l < count && higherPriority(data[l], data[best])) best = l;
        if (r < count && higherPriority(data[r], data[best])) best = r;
        if (best == i) break;
        swapAt(i, best);
        i = best;
```

```
    }
  }

  // prevent accidental copying of the raw array
  MaxHeap(const MaxHeap&);
  MaxHeap& operator=(const MaxHeap&);

public:
  MaxHeap() : count(0), capacity(16) { data = new Email[capacity]; }
  ~MaxHeap() { delete[] data; }

  bool isEmpty() const { return count == 0; }
  int  size()    const { return count; }

  void insert(const Email& e) {
    if (count == capacity) grow();
    data[count] = e;
    siftUp(count);
    count++;
  }

  // Caller must check isEmpty() first.
  const Email& peek() const { return data[0]; }

  // Caller must check isEmpty() first.
  Email extractMax() {
```

```
        Email top = data[0];
        count--;
        if (count > 0) {
            data[0] = data[count];
            siftDown(0);
        }
        return top;
    }
};

// ------------------------------------------------------------ Helpers ----
string trim(const string& s) {
    size_t start = 0, end = s.size();
    while (start < end && (s[start] == ' ' || s[start] == '\t' ||
                           s[start] == '\r' || s[start] == '\n')) start++;
    while (end > start && (s[end - 1] == ' ' || s[end - 1] == '\t' ||
                           s[end - 1] == '\r' || s[end - 1] == '\n')) end--;
    return s.substr(start, end - start);
}

// Returns 0-4 for a valid category, -1 otherwise.
int categoryRank(const string& cat) {
    if (cat == "Boss")          return 0;
    if (cat == "Subordinate")   return 1;
    if (cat == "Peer")          return 2;
    if (cat == "ImportantPerson") return 3;
```

```
    if (cat == "OtherPerson")    return 4;
    return -1;
}

// Converts MM-DD-YYYY to YYYYMMDD. Returns -1 if malformed.
int dateToKey(const string& date) {
    int mm, dd, yyyy;
    char extra;
    if (sscanf(date.c_str(), "%d-%d-%d%c", &mm, &dd, &yyyy, &extra) != 3) return -1;
    if (mm < 1 || mm > 12 || dd < 1 || dd > 31 || yyyy < 0) return -1;
    return yyyy * 10000 + mm * 100 + dd;
}

// Parses the part of an EMAIL command after "EMAIL ". Returns false if invalid.
bool parseEmail(const string& args, Email& out) {
    size_t c1 = args.find(',');
    if (c1 == string::npos) return false;
    size_t c2 = args.find(',', c1 + 1);
    if (c2 == string::npos) return false;

    out.sender  = trim(args.substr(0, c1));
    out.subject = trim(args.substr(c1 + 1, c2 - c1 - 1));
    out.date    = trim(args.substr(c2 + 1));

    out.rank    = categoryRank(out.sender);
    out.dateKey = dateToKey(out.date);
```

```
    return out.rank >= 0 && out.dateKey >= 0;
}

// ---------------------------------------------------------------- Main ----
int main(int argc, char* argv[]) {
    string filename;
    if (argc > 1) {
        filename = argv[1];
    } else {
        cout << "Enter input file name: ";
        getline(cin, filename);
        filename = trim(filename);
    }

    ifstream in(filename.c_str());
    if (!in.is_open()) {
        cout << "Error: could not open file \"" << filename << "\"." << endl;
        return 1;
    }

    MaxHeap inbox;
    long arrival = 0;
    string line;

    while (getline(in, line)) {
        line = trim(line);
```

```
if (line.empty()) continue;


if (line.compare(0, 6, "EMAIL ") == 0) {
   Email e;
   if (parseEmail(line.substr(6), e)) {
      e.seq = arrival++;
      inbox.insert(e);
   } else {
      cout << "Warning: skipping invalid EMAIL command: " << line << endl;
   }
}
else if (line == "NEXT") {
   if (inbox.isEmpty()) {
      cout << "There are no emails to read." << endl;
   } else {
      const Email& e = inbox.peek();
      cout << "Next email:" << endl;
      cout << "Sender: "  << e.sender  << endl;
      cout << "Subject: " << e.subject << endl;
      cout << "Date: "    << e.date    << endl;
   }
}
else if (line == "READ") {
   if (inbox.isEmpty()) {
      cout << "There are no emails to read." << endl;
   } else {
```

```
            inbox.extractMax();
        }
      }
      else if (line == "COUNT") {
        cout << "There are " << inbox.size() << " emails to read." << endl;
      }
      else {
        cout << "Warning: unrecognized command: " << line << endl;
      }
    }

    return 0;
}
```

CoPilot, accessed through desktop app, code:

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <string>
using namespace std;
struct Email {
   string sender;
   string subject;
   string date;
};
class MaxHeap {
private:
   vector<Email> heap;
   int getPriority(const string& sender) {
      if (sender == "Boss") return 5;
      if (sender == "Subordinate") return 4;
      if (sender == "Peer") return 3;
      if (sender == "ImportantPerson") return 2;
      return 1; // OtherPerson
   }
   int dateValue(const string& date) {
      int month, day, year;
      char dash;
      stringstream ss(date);
```

```
    ss >> month >> dash >> day >> dash >> year;
    return year * 10000 + month * 100 + day;
}
bool higherPriority(const Email& a, const Email& b) {
    int p1 = getPriority(a.sender);
    int p2 = getPriority(b.sender);
    if (p1 != p2)
        return p1 > p2;
    return dateValue(a.date) > dateValue(b.date);
}
void heapifyUp(int index) {
    while (index > 0) {
        int parent = (index - 1) / 2;
        if (higherPriority(heap[index], heap[parent])) {
            swap(heap[index], heap[parent]);
            index = parent;
        }
        else {
            break;
        }
    }
}
void heapifyDown(int index) {
    int size = heap.size();
    while (true) {
        int left = 2 * index + 1;
```

```
        int right = 2 * index + 2;
        int largest = index;
        if (left < size &&
            higherPriority(heap[left], heap[largest])) {
            largest = left;
        }
        if (right < size &&
            higherPriority(heap[right], heap[largest])) {
            largest = right;
        }
        if (largest != index) {
            swap(heap[index], heap[largest]);
            index = largest;
        }
        else {
            break;
        }
      }
    }
public:
    void insert(const Email& email) {
        heap.push_back(email);
        heapifyUp(heap.size() - 1);
    }
    bool empty() const {
        return heap.empty();
```

```
    }
    int size() const {
        return heap.size();
    }
    Email top() const {
        return heap[0];
    }
    void pop() {
        if (heap.empty())
            return;
        heap[0] = heap.back();
        heap.pop_back();
        if (!heap.empty())
            heapifyDown(0);
    }
};
Email parseEmailLine(const string& line) {
    Email email;
    string content = line.substr(6); // remove "EMAIL "
    stringstream ss(content);
    getline(ss, email.sender, ',');
    getline(ss, email.subject, ',');
    getline(ss, email.date);
    auto trim = string& s {
        while (!s.empty() && s.front() == ' ')
            s.erase(0, 1);
```

```
        while (!s.empty() && s.back() == ' ')
            s.pop_back();
    };
    trim(email.sender);
    trim(email.subject);
    trim(email.date);
    return email;
}
int main() {
    string filename;
    cout << "Enter input file name: ";
    cin >> filename;
    ifstream infile(filename);
    if (!infile) {
        cout << "Error opening file.\n";
        return 1;
    }
    MaxHeap inbox;
    string line;
    while (getline(infile, line)) {
        if (line.rfind("EMAIL ", 0) == 0) {
            Email email = parseEmailLine(line);
            inbox.insert(email);
        }
        else if (line == "COUNT") {
            cout << "There are "
```

```
        << inbox.size()
        << " emails to read.\n";
    }
    else if (line == "NEXT") {
      if (inbox.empty()) {
        cout << "No emails remaining.\n";
      }
      else {
        Email email = inbox.top();
        cout << "Next email:\n";
        cout << "Sender: "
            << email.sender << '\n';
        cout << "Subject: "
            << email.subject << '\n';
        cout << "Date: "
            << email.date << '\n';
      }
    }
    else if (line == "READ") {
      if (inbox.empty()) {
        cout << "No emails remaining.\n";
      }
      else {
        inbox.pop();
      }
    }
```

```
    }
    infile.close();
    return 0;
}
```

Analysis:

Correctness:

| Claude | CoPilot |
|---|---|
| Works as intended, no major flaws to be noted in terms of correctness | Would not compile initially, I had to ask copilot to fix its error. Afterwards it did compile and run as expected. |

Time complexity:

| function | Claude | CoPilot |
|---|---|---|
| Email | O(logN) | O(logN) |
| Read | O(logN) | O(logN * M) M = date length because it re computes the date length every single time |
| Next | O(1) | O(1) |
| Count | O(1) | O(1) |
| Heap comparisons | O(1) | O(M) |

Space complexity:

| Claude | CoPilot |
|---|---|
| Slightly larger email struct | Slightly smaller email struct |
| Keeps allocated space even after emails are read | Keeps allocated space even after emails are read |

Maintainability:

| Claude | CoPilot |
|---|---|
| Included some comments, although limited | Included practically no comments |

Choice and modifications:

Decision:

I chose to use the code by Claude because I felt it was more maintainable because of the improved comments, it had better time complexity in most functions because of the lack of unnecessary computations that CoPilot had and had no major space complexity issues.

Modifications:

- Change the “default” value of priority from -1 to 5 if there if the sender does not match one of the predetermined categories
- Add comments to improve readability